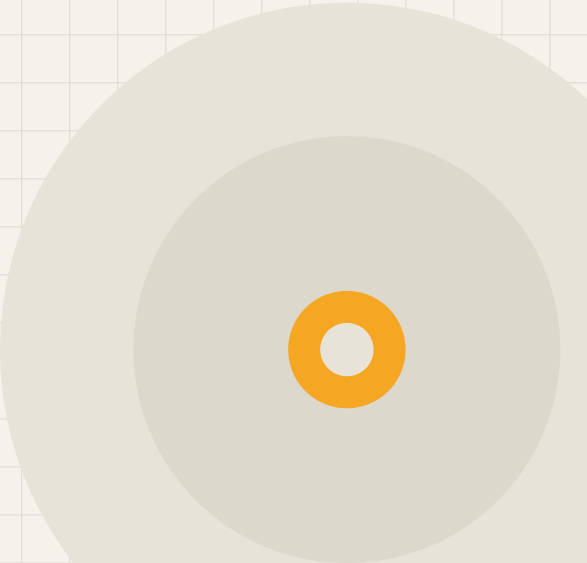
IntoData KB

Integration Intelligence Platform

Technical Architecture · v3.0 · May 2026

- **Code Intelligence**
  Scan & document integration codebases automatically
- **Knowledge Q&A**
  RAG-powered search over accumulated project knowledge
- **Provider-Agnostic AI**
  Swappable LLM & embedding backends — no vendor lock-in
- **Enterprise-Ready**
  RBAC, audit logs, usage tracking, one VPS per client



Platform Vision

What IntoData KB does — and why it exists

Integration projects generate tacit knowledge that disappears when the project ends.

Source code (Talend, Workato, Lobster, ...) is opaque. Design decisions live in people's heads.

1 Observe

The platform scans integration codebases and infers structure, lineage and intent — automatically.

No manual tagging. No extra steps.

2 Accumulate

Generated docs, decisions and lessons are ingested into a vector knowledge base per project.

Every project makes the next one smarter.

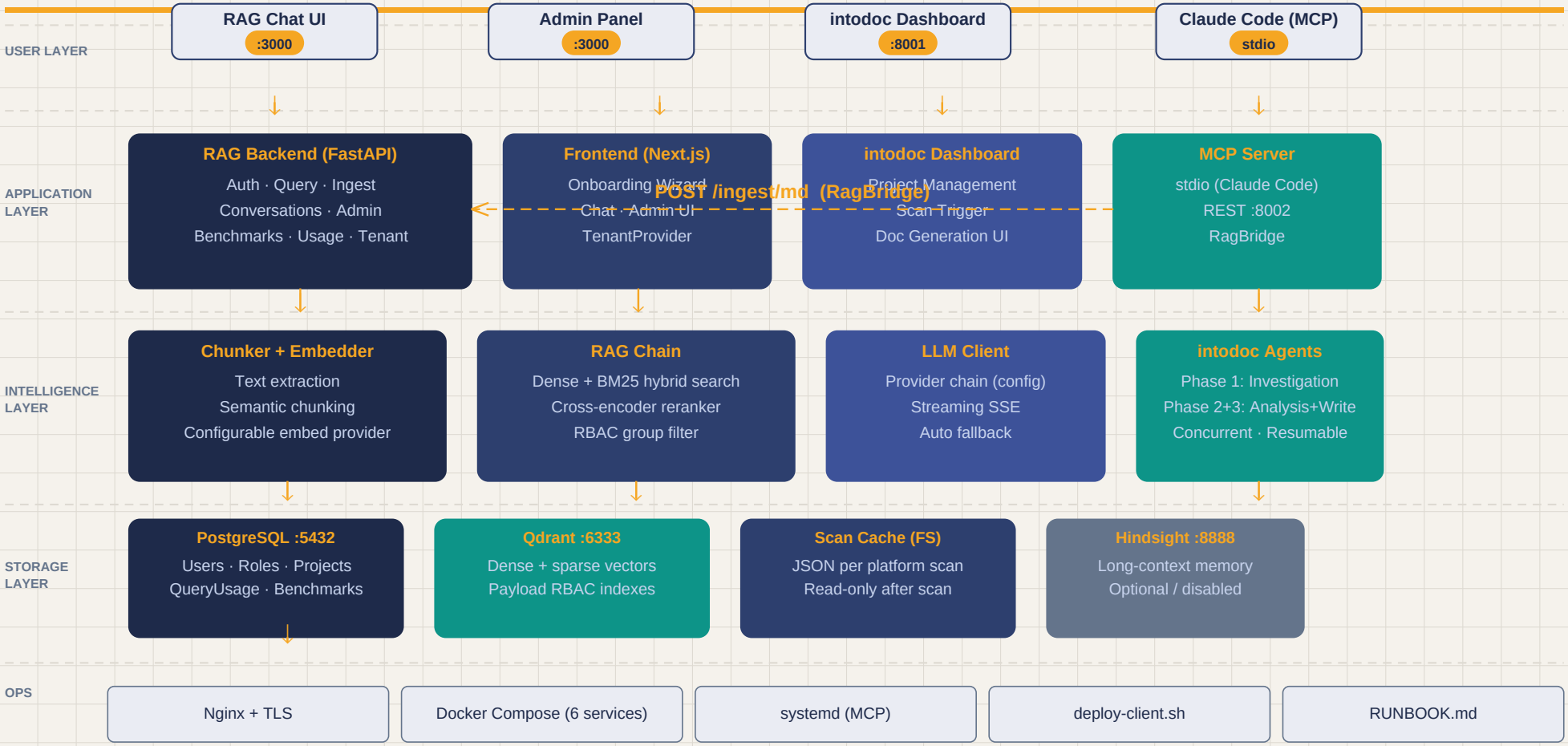
3 Answer

Consultants ask natural-language questions and receive grounded answers with source citations.

The right knowledge, at the right moment.

Full System Architecture

All components, services and the data paths that connect them



Subsystem A — intodoc: Code Intelligence

Turns opaque integration source code into structured, searchable documentation

Source Code
Talend XMI / Workato JSON
Lobster XML / Generic source

↓

Scanner CLI
Platform-specific parser
Credential redaction built-in
Normalized IR component model

↓

JSON Cache
intodoc-output/ per project
Jobs · groups · lineage
Read-only after scan

↓

Phase 1: Investigate
LLM reads scan cache
Answers structured fact questions
Builds domain context

↓

Phase 2: Analyse
Group-level LLM agents (parallel)
Cross-group quality scorecard
Merged for large groups

↓

Phase 3: Write
Per-document LLM agents
Concurrent · resumable
docs_staging/ → docs/ (atomic swap)

↓

Markdown Docs
Structured output · ASCII-safe
RagBridge pushes to RAG
Versioned per GenerateRun

Multi-Platform Architecture

One adapter per platform (Talend, Workato, Lobster, Generic),
each producing the same normalised IR. Adding a platform = one adapter.

Four-Phase AI Generation

Phase 1 builds domain context. Phase 2 analyses groups (parallel agents).
Phase 2.6 runs a cross-group quality scorecard. Phase 3 writes docs.

Custom Agent Runtime

Bounded agent harness — no external orchestration framework.
Providers are config-driven: custom, openai-compat, anthropic, ollama.

Atomic Staging Swap

Docs written to docs_staging/ during generation.
Server swaps docs/ only when all phases succeed — no partial state.

Credential Safety

Two-layer redaction (regex + structural) before any content
reaches an LLM. No secrets sent to AI providers.

Subsystem B — RAG Pipeline: Knowledge Q&A

From raw documents to grounded, project-scoped answers

Ingestion Path

Document Input
PDF / DOCX / PPTX / XLSX / MD
Manual upload or RagBridge POST

↓

Parse & Chunk
Format-specific text extraction
Semantic chunking (boundary-aware)
Fixed fallback: ~500 tokens + overlap

↓

Embed
Dense: configurable embedding provider
Sparse: BM25 keyword vectors
Dimension-aware collection mgmt

↓

Qdrant Upsert
Dense + BM25 sparse vectors stored
Payload: allowed_groups, doc_id, project_id
Indexed for fast RBAC filter

↓

Postgres Record
Document row: filename, uploader
chunk_count, project_id, timestamps
IngestionJob tracks status

Query Path

User Question
POST /api/v1/query or /query/stream
JWT auth (httpOnly cookie)
project_id + intent classification

↓

Memory Recall
Hindsight: user memories (optional)
Quality insights, past corrections

↓

Hybrid Search
Dense cosine + BM25 sparse (parallel)
RBAC filter injected server-side

↓

Rerank
Cross-encoder re-scores top candidates
Min score threshold applied

↓

LLM Generate
Context + memory + history assembled
Configurable provider chain generates

↓

Usage Persist
Tokens · cost · latency · model
QueryUsage row in Postgres
Dashboard-visible per project

The Provider-Agnostic AI Layer

LLM and embedding providers are runtime configuration — the platform is not tied to any vendor

RAG Chain
Takes query vector
Calls Qdrant search
Assembles context

LLM Client
Tries provider[0]
Falls back to [1..n]
Streams tokens via SSE

Embedder
Tries provider[0]
Falls back to [1..n]
Returns float[] vector

intodoc Agents
Phase 1/2/3 orchestration
Custom agent runtime
Calls LLM Client internally

LLM Provider Pool (config-driven)
OpenAI-compatible APIs
Anthropic API
Local Ollama models
Any compatible interface

Embedding Provider Pool (config-driven)
Cloud embedding APIs (high-dim)
Local Ollama embedding models
BM25 sparse (always on)
Swappable — no downstream changes

Provider Chain Mechanics:
Provider lists are ordered in config. On failure, timeout or error the next provider is tried automatically — no code change required.
Qdrant collection auto-recreates if embedding dimension changes. Usage tracking fires regardless of which provider answered.

Security Architecture — RBAC & Auth

Project-scoped access enforced from API gateway to vector search

Authentication & Session Flow

Login Request
Email + password submitted
Rate limited: 5 req/min

↓

Credential Verify
bcrypt hash compare
DB lookup, is_active check

↓

MFA Challenge (opt.)
If TOTP enabled:
pyotp.verify() → mfa_token JWT

↓

Token Pair Issued
Access JWT (60 min TTL)
Refresh token (SHA-256 hash in DB)

↓

httpOnly Cookies Set
Tokens set as httpOnly cookies
XSS-resistant, SameSite=lax

↓

RBAC Middleware
Every request: decode JWT
Load roles + 9 permissions
Enforced before handler

RBAC Model

Roles

admin	Full platform: user mgmt, all projects, system config
manager	Manage projects, assign users, view usage dashboards
developer	Query + ingest within explicitly assigned projects only

Vector Search Filter (every query)

Every chunk stores allowed_groups in its Qdrant payload.
Every query applies must filter: user's groups ∩ chunk's groups ≠ ∅
Users in group 'project-A' cannot see chunks from 'project-B'.
RBAC enforced at Qdrant level — not post-hoc Python filtering.

9 Discrete Permissions

query_rag	ingest_documents	manage_users
manage_roles	assign_projects	view_all_projects
manage_projects	view_audit_logs	view_assigned_projects

Data & Storage Architecture

What lives where — and the design rationale behind each store

PostgreSQL 16

:5432

Relational truth — identity, access control, usage, benchmarks

Schema / contents

users
id, email, hashed_password, role, totp_secret, is_active
refresh_tokens
token_hash (SHA-256), user_id, expires_at, revoked
projects
id, name, slug, allowed_groups[]
project_assignments
user_id ↔ project_id (M2M)
documents
id, filename, project_id, chunk_count, uploaded_by
conversations
id, user_id + conversation_messages[]
query_usage
tokens, cost_usd, latency_ms, model, provider, project_id
benchmark_runs
question, answer, precision@5, mrr, llm_judge_score

Design rationale

ACID for identity & billing.
Async SQLAlchemy.
Schema managed by Alembic.
Both RAG and MCP subsystems share this instance.

Qdrant

:6333/6334

Vector similarity search — dense semantic + sparse BM25 per chunk

Schema / contents

collection (per project/tenant)
dense vector (semantic similarity)
sparse BM25 vector (keyword precision)
payload
chunk_text, doc_id, project_id, allowed_groups[]
indexes
allowed_groups (keyword), document_id (integer)
project_id (keyword) for scoped search
Collection auto-recreated on dimension change

Design rationale

Hybrid search outperforms pure semantic on technical queries.
Payload RBAC filter eliminates cross-project leakage at DB level, not applicat

Scan Cache (filesystem)

intodoc-output/

Normalised intermediate representation from the intodoc scanner CLI

Schema / contents

per-project JSON
jobs, components, groups, lineage
platform metadata normalised to common IR
redaction_report.json alongside scan_result.json
read-only after scan; re-scan overwrites
docs_staging/
agent output staging area (atomic swap on success)
docs/
final Markdown — replaced only on full-phase success

Design rationale

Filesystem cache decouples scanner from doc generation.
Staging swap guarantees users always see a complete, consistent doc set per GenerateRu

Multi-Client Tenancy & Deployment Model

One VPS per client — full isolation through deploy-client.sh

Per-Client Isolation Guarantees

PostgreSQL Database
Separate DB: intodata_kb_{client}
No cross-client data access

Qdrant Collection
Scoped collection: {client}_documents
Vectors fully isolated

Secrets & JWT
Randomised per-client: JWT_SECRET_KEY
POSTGRES_PASSWORD, MCP_INGEST_KEY

nginx + TLS
Separate server block per client
Individual certbot cert per domain

Docker Compose Project
docker compose -p {client}
Prefixes all containers, networks, volumes

MCP systemd Instance
mcp-server@{client}.service
Instance per client on host

deploy-client.sh Workflow

1. Invoke Script
./deploy/deploy-client.sh
--client-name acme --port-offset 0
--base-domain nexus.intodata.nl

↓

2. Generate Secrets
Randomise JWT_SECRET_KEY
POSTGRES_PASSWORD, MCP_INGEST_KEY
Write /etc/intodata-kb/envs/acme.env

↓

3. Render nginx Block
envsubst on nexus-client.conf.template
Write /etc/nginx/sites-available/nexus-acme.conf
(nginx -t to validate)

↓

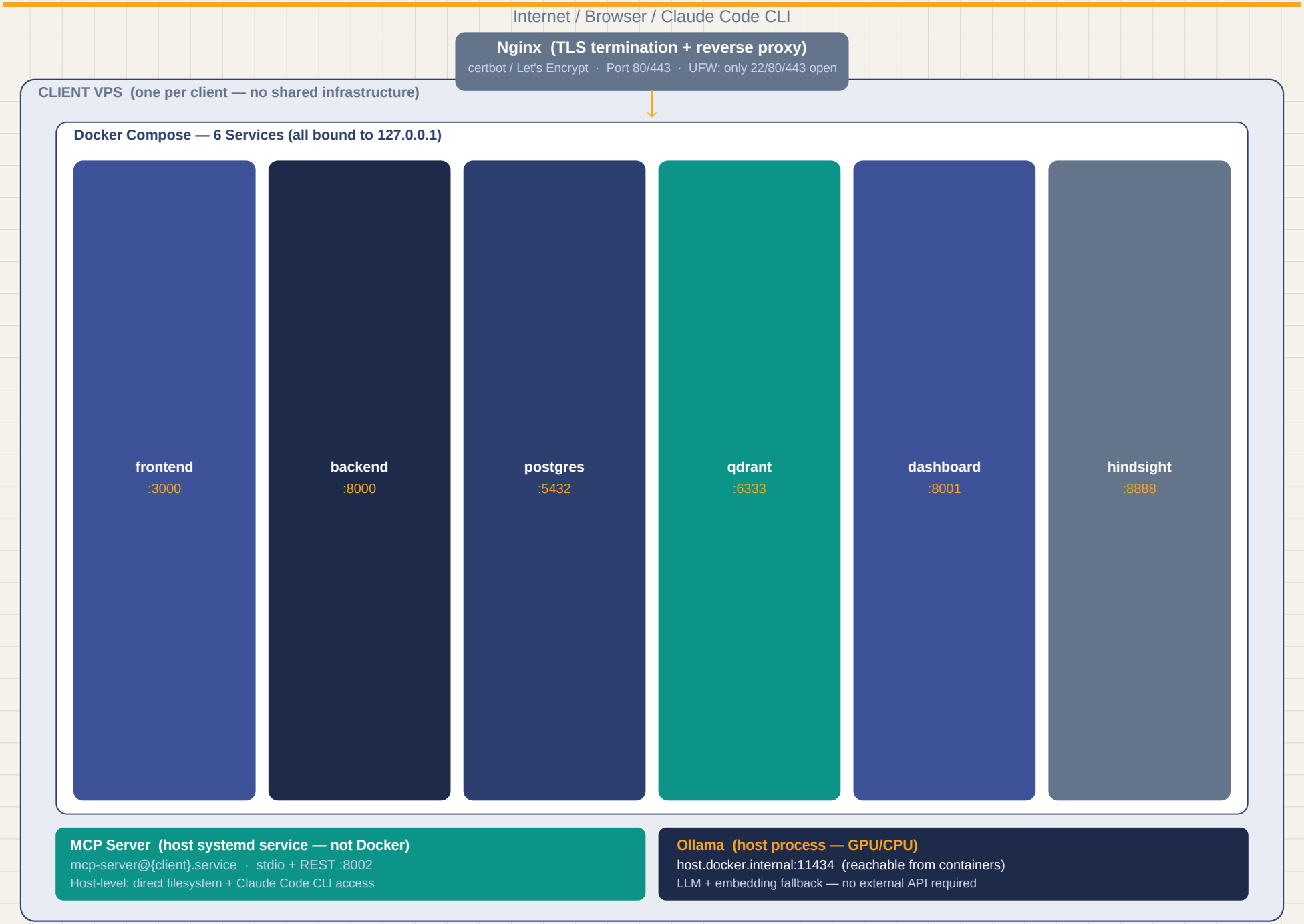
4. Launch Stack
docker compose --env-file acme.env -p acme up -d
Health checks gate service startup
certbot obtains Let's Encrypt cert

↓

5. Start MCP Service
systemctl enable/start mcp-server@acme
REST API :8002 available on host
Claude Code connects via stdio

Deployment Topology

Client VPS — services, host processes and traffic routing



End-to-End Knowledge Flow

How raw integration source code becomes an answerable knowledge base

1 — Integration Source Code
Talend / Workato
Lobster / Generic

2 — intodoc Scanner
Platform parser
Credential redaction
Normalized JSON cache

3 — AI Doc Agents (4 phases)
Investigate · Analyse
Quality check · Write
Concurrent · resumable

4 — Markdown Documents
Structured output
ASCII-safe
Per-project

5 — RAG Ingestion
Parse · chunk · embed
Dense + sparse vectors
Qdrant upsert

6 — Knowledge Base
RBAC-filtered
hybrid vector search
Project-scoped

7 — Consultant Q&A
Natural language query
Grounded answer + SSE
Source citations

Accumulation Principle

Every project adds to the shared vector store. Later projects benefit from earlier ones — decisions, patterns and lessons searchable across the portfolio.

Zero-Overhead Capture

Consultants do not tag, categorise or manually document. The scanner observes code; the AI agent writes the docs; the platform ingests them. Zero extra steps.

API Surface

RAG Backend (:8000) and MCP Server (:8002) — route groups, purpose and access level

Auth /api/v1/auth

`POST /login` — public
Credential verify → JWT pair in httpOnly cookies

`POST /logout` — auth
Revoke refresh token, clear cookies

`POST /refresh` — auth
Rotate access + refresh token pair

`POST /verify-mfa` — mfa_token
Exchange mfa_token for full session after TOTP

`POST /bootstrap-admin` — open-once
Seed first admin (one-time, disabled after)

Query /api/v1/query

`POST /query` — developer+
Full RAG chain: embed → search → rerank → generate

`POST /query/stream` — developer+
Same chain, SSE streaming response token-by-token

`POST /query/debug` — developer+
Returns retrieved chunks + scores pre-generation

`POST /conversations` — developer+
Multi-turn conversation session management

`GET /projects` — developer+
RBAC-filtered project listing for scoping

Ingest /api/v1

`POST /documents` — developer+
Upload PDF/DOCX/PPTX/XLSX → chunk → embed → Qdrant

`POST /ingest/md` — ingest-key
Ingest Markdown string (RagBridge from MCP)

`GET /documents` — developer+
List documents in a project (RBAC-filtered)

`DELETE /documents/:id` — manager+
Remove document + all Qdrant vectors (cascade)

Admin /api/v1/admin

`CRUD /users` — admin
User management: create, update, deactivate

`CRUD /roles` — admin
Role + permission management

`POST /assignments` — manager+
Assign user to project (project_assignments)

`GET /audit-logs` — admin
Full audit trail of platform actions

`GET /usage/summary` — manager+
Aggregated token + cost data per project/model

System & KB /api/v1

`GET /tenant/config` — public
Public: name, logo, colors for TenantProvider

`GET /health` — public
DB, Qdrant, Ollama, Hindsight health check

`POST /benchmark/run` — manager+
Run benchmark Q&A set against RAG chain

`GET/DEL /memory/:uid` — admin
Manage Hindsight long-context memory per user

`GET /onboarding/config` — auth
Wizard config for initial setup flow

MCP Server :8002

`GET/POST /mcp/projects` — jwt
List / register code repositories

`POST /mcp/scan` — jwt
Trigger intodoc scan for a project

`POST /mcp/generate` — jwt
Start 4-phase doc generation (custom runtime)

`GET /mcp/generate-runs/:id` — jwt
Poll doc generation run status + progress

`GET /logs/stream` — jwt
SSE real-time log stream during scan/generation

IntoData KB — Architecture at a Glance

Code Intelligence

intodoc scans Talend, Workato, Lobster and generic integration platforms.
Four AI phases: investigate, analyse, quality-check, write — concurrent & resumable.
Markdown output is atomic — staging swap ensures a consistent doc set per run.

RAG Knowledge Q&A

Hybrid vector search: dense semantic + BM25 sparse, cross-encoder reranked.
Every query is RBAC-filtered at the Qdrant payload level (allowed_groups).
Answers stream in real-time via SSE with source citations + usage tracking.

Provider-Agnostic AI

LLM and embedding providers are ordered config chains — no vendor lock-in.
Automatic fallback: if provider[0] fails, provider[1] is tried — no code change.
Swapping or adding a provider requires only an env var / config update.

Enterprise Operations

Usage tracked per call: tokens, cost, latency, model, provider, project.
One VPS per client — deploy-client.sh generates isolated env, nginx, Docker stack.
Reproducible delivery: RUNBOOK.md + SMOKE-TEST.md gate every deployment.